The 21st Century Lab Notebook

Guide of Best Practices

Sergi Casado Prieto

June 2026

Contents

The 21st Century Lab Notebook

Guide of Best Practices for Research Documentation

> A practical guide for students and researchers entering a laboratory for the first time. No prior knowledge of Git, YAML, or documentation theory required.

Version: 1.3 **Author:** Sergi Casado Prieto - IMB-CNM-CSIC / UAB
Date: June 2026
Repository: https://github.com/sergicasadoprieto-ctrl/tfg-quadern-laboratori-21c

---

Who is this guide for?

This guide is for you if:

- You are starting a new research project and have no documentation system in place
- You have been using paper notebooks or Word documents and want something better
- You want your work to be reproducible, searchable, and traceable without spending hours on formatting

You do not need to know Git, YAML, or any programming language to use this framework. Everything in this guide uses free or open-source tools that run on any operating system.

What this guide covers

1. Why documentation matters
2. The tools you need
3. How to start a new project
4. How to document day-to-day
5. How to use AI assistance (optional)
6. How to maintain the repository
7. Project closure checklist

1. Why Documentation Matters

The problem

You finish a six-month project. Six months later, a colleague asks you to reproduce one of your results. You open your notes and find:

- A folder called `final_result_v2_NEW_fixed`
- A paper notebook with handwriting you can barely read
- An email chain with parameters scattered across twelve messages
- A Word document with no dates, no version history, and no explanation of why you made the decisions you made

This is not an unusual situation. During the development of QL21, similar issues appeared repeatedly when reviewing older project records and laboratory notes.

What good documentation looks like

Good documentation is not about writing more. It is about writing the right things in the right place, consistently, from day one.

A well-documented project has:

- A clear record of what was done, when, and by whom
- The reasoning behind every significant decision
- All experimental parameters, not just the ones that worked
- A version history that shows how the project evolved
- A structure that anyone can navigate without asking you for help

The cost of starting late

Documentation is one of those things that is easy to defer and very expensive to reconstruct. Every day you wait, the cost of catching up increases. Parameters you remember today will be gone in two weeks. Context that seems obvious now will be opaque in six months.

The single most effective thing you can do is start on day one, with a structure already in place, so that documentation is part of the workflow rather than an addition to it.

2. The Tools You Need

The QL21 workflow can be understood as a three-layer stack. The first layer contains the core infrastructure required for traceability and reproducibility. The second layer contains the tools used to create and manage documentation. The third layer consists of optional AI assistants that may support documentation tasks.
Layer 1 - Structure (Required)

These are the core components of the framework. They define how information is stored and tracked.

Markdown
- Plain text format used for logs, notes, reports, and project documentation.
- Human-readable, future-proof, and compatible with virtually any editor.

YAML
- Metadata format used in document frontmatter.
- Stores information such as author, date, version, project identifier, and document type.
- YAML does not record activity. It only describes the current state of the project.

In this framework YAML is split into four roles:
- **user** - who is working on the project
- **project** - what the project is
- **governance** - rules and constraints
- **branding** - visual/identity layer (optional)

Git
- Version control system used to track changes over time.
- Provides a complete history of the project and enables full traceability.

BibTeX
- Standard reference format used for managing bibliographic information.
- Integrates with Zotero and LaTeX workflows.

LaTeX
- Typesetting system commonly used for academic writing.
- Recommended for theses, reports, and scientific publications.

Layer 2 - Tools (Recommended)

The framework can be used with many applications. The tools below are those used during the development of QL21.

Obsidian

- Primary environment for writing and organising Markdown documentation.
- Supports YAML frontmatter, internal links, graph visualisation, and fast search.

GitHub

- Remote repository hosting and backup platform.
- Facilitates collaboration and publication through GitHub Pages.

Zotero

- Reference manager used to collect and organise scientific literature.
- Exports bibliographic records in BibTeX format.

Visual Studio Code

- General-purpose editor with strong support for Markdown, YAML, Git, and Python.

Pandoc

- Document conversion tool.
- Converts Markdown into PDF, Word, HTML, and LaTeX formats.

Marp

- Presentation framework based on Markdown.
- Useful for creating seminar and conference slides directly from project documentation.

Overleaf

- Online LaTeX editor and compiler.
- Useful when collaborating on theses or scientific manuscripts.

Python

- Optional scripting language for data processing, automation, and analysis.

Power BI

- Optional dashboarding tool for survey analysis and project metrics.

Layer 3 - AI Assistance (Optional)

AI tools are not required to use QL21. When permitted by institutional policy, they may help generate summaries, structure notes, draft documentation, or prepare reports.

Examples include:

- Ollama (local models)
- ChatGPT
- Claude
- Gemini
- Copilot
- Perplexity
- DeepSeek
- Grok
- Mistral

Researchers remain responsible for the accuracy, interpretation, and scientific validity of all project documentation.

What You Do Not Need

- A subscription to any service
- A high-performance computer
- Programming experience
- Continuous internet access

All core QL21 documents are plain text files and can be created, edited, and versioned offline.

How QL21 structures information

QL21 separates documentation into two complementary layers:

Project configuration (YAML files) define the static structure of the project: who is involved, what the project is, what rules apply, and how it looks. These files change rarely.

Project documentation (Markdown files) record what happens: daily experimental activity (LOG), ideas and insights (ZN), weekly summaries (RPT), and project documents (PROJ). These files change continuously and form the complete history of the work.

> Think of it this way:
> - YAML = the “settings panel” of the project
> - Markdown = the “project diary”

Using with institutional systems

This framework is designed for Git-based workflows. If your institution already uses a self-hosted Git server (GitLab, Gitea, or similar), the framework works without any modification. Clone the toolkit locally and redirect the remote:

```
git remote set-url origin https://your-institution-server/your-project.git
git push -u origin main
```

If your institution uses a document management system such as OpenText or SharePoint, these platforms serve a different purpose than Git and can coexist. Use Git for daily documentation and traceability, and export final deliverables to your institutional system when required for formal approval or archival.

3. How to Start a New Project

Depending on your familiarity with Git, the initial setup may take longer. Most users find that the workflow becomes routine after a few sessions.

Step 1: Create the repository

Create a repository on GitHub (or an equivalent Git hosting platform used by your institution).
1. Go to https://github.com and log in
2. Click **New repository**
3. Name it following your project acronym, for example: `my-project-2026`
4. Set it to **Public** or **Private** depending on your policy
5. Do NOT add a README, .gitignore, or licence (you will create these manually)
6. Click **Create repository**
7. Copy the repository URL shown on the next page

Step 2: Clone the repository locally

Open a terminal (or GitHub Desktop) and run:

```
git clone https://github.com/username/my-project-2026.git
cd my-project-2026
```

Step 3: Create the folder structure

```
mkdir 00-governance
mkdir 01-literature
mkdir 02-framework
mkdir 03-pilot
mkdir 04-outputs
mkdir 05-journal
```

Each folder has a specific purpose:

Folder	Purpose
00-governance	Manifests, SOW, confidentiality agreements
01-literature	Papers, notes, references.bib
02-framework	Templates and naming convention
03-pilot	Laboratory logs, ZN notes, reports
04-outputs	Final deliverables
05-journal	Personal notes, session summaries

The numerical prefixes are simply there to keep the folders in a stable order across systems

Step 4: Copy the templates

Copy the four core templates from the QL21 kit to `02-framework/templates/`:

Template	Purpose
TPL-LOG.md	Laboratory log
TPL-ZN.md	Zettel note
TPL-RPT.md	Weekly report
TPL-PROJ.md	Project initialisation

Note: TPL files are templates. They are never used directly during the project. Each session creates a new instance (LOG, ZN, RPT, etc.) based on these templates. Every operational file is always an instantiated copy of a TPL.

Step 5 (Optional): Fill in the governance manifests

This step is only necessary if you plan to use AI assistants as part of your documentation workflow. The manifests provide structured project context that can be shared with an AI system to help reconstruct project information between sessions, they are optional but useful for reproducibility and context reconstruction.

If you plan to use AI assistance, copy the governance manifests from the QL21 kit to `00-governance/` and complete the relevant fields. If you do not intend to use AI assistance, you may skip this step and continue directly to Step 6.

[[user-manifest-template.yaml]]

```yaml
name: "Your Full Name"
acronym: "YFN"
jobTitle: "Your role"
worksFor:
  name: "Your institution"
projectContext:
  name: "Your project name"
  startDate: "YYYY-MM-DD"
  supervisor:
    name: "Supervisor name"
    affiliation: "Institution"
```

[[project-manifest-template.yaml]]

```yaml
name: "Project full name"
acronym: "PROJECT-ACRONYM"
description: >
  One paragraph describing what this project is about and what problem it
↪  addresses.
governance:
  statementOfWork: "SOW-filename"
  scopeRule: "If it is not in the SOW, it is out of scope."
```

[[governance-manifest-template.yaml]]

```yaml
  project:
    name: "[Full project name]"
```

```
    acronym: "[ACRONYM]"
    institution: "[Institution(s)]"
    type: "[e.g. Methodological Research / Software Development /
↪   Operational]"
    domain: "[e.g. Scientific Documentation / Data Engineering]"
```

[[branding-manifest-template.yaml]] (optional)

Fill in your preferred colours and fonts, or leave the QL21 defaults as they are.

Step 6: Create the README and CHANGELOG

README.md - minimum content:

```
# Project Name

Short description of the project.

## People
- Student: Your name
- Supervisor: Supervisor name
- Institution: Your institution

## Structure
Standard QL21 folder structure.
```

CHANGELOG.md — minimum content:

```
# Changelog

## [0.1.0] — YYYY-MM-DD

### Added
- Initial repository structure
- Governance manifests v1.0
- Core templates
```

Step 7: First commit

```
git add .
git commit -m "feat: initialise project structure v0.1.0"
git push -u origin main
```

At this point the repository structure is in place and documentation can begin immediately. The remainder of the guide focuses on maintaining this structure throughout the project lifecycle. From this point, every session follows the daily workflow described in Section 4.

4. How to Document Day-to-Day

This section describes how project history is generated from the static structure defined in the YAML files.

Before the session

Open the last CHANGELOG entry or the most recent ZN note in [[05-journal]]. Read it. This reconstructs the context of where you left off without having to open any other file.

If you are using AI assistance, load the three governance manifests and the last ZN file before starting. See [[Section 5]] (AI Assistance) for details.

During the session: the LOG file

Every laboratory session, coding session, or significant work block gets a LOG file. Create one by copying `TPL-LOG.md` to `03-pilot/logs/` and renaming it following the naming convention:

LOG-{YYYYMMDD}T{HHmmss}-{AUTHOR}-1.00.md

Example:

LOG-20260114T090000-ABC-1.00.md

Fill in the YAML frontmatter first:

```
---
id: LOG-{YYYYMMDD}T{HHmmss}-{AUTHOR}-1.0
type: LOG
date:
author:
project:
phase:
deliverable:
tags:
related_logs:
related_notes:
related_deliverables:
```

```
status: Template
---
```

Then write the body of the log following the six sections:

1. Objective
One sentence. What did you want to achieve today?

2. Activity
What did you do, step by step? Be specific. Include equipment names, parameter values, sample IDs, and software versions. Do not summarise. Write it as you would tell a colleague who needs to repeat the exact same procedure tomorrow.

3. Observations
What did you see? Include unexpected results, anomalies, and anything that surprised you. Do not interpret yet. Just record.

4. References
Links to related LOG files, ZN notes, or external sources.

When you have an insight: the ZN file

A Zettel Note captures one idea, observation, or insight. It is not a log of what you did. It is an atomic unit of knowledge that can connect to other notes and grow over time.

Create a ZN file by copying `TPL-ZN.md` to `05-journal/` and renaming it:

ZN-{YYYYMMDD}T{HHmmss}-{YOUR_ACRONYM}-1.00.md

The rule is: one ZN, one idea. If you need to write about two ideas, create two ZN files. This keeps each note focused and searchable.

Good candidates for a ZN:

- An unexpected result that needs further investigation
- A concept from a paper that is relevant to your project
- A decision you made and the reasoning behind it
- A connection between two things you had not noticed before
- ZNs are non-linear and can be created at any time, independently of sessions.

At the end of the week: the RPT file

Every week, create a weekly report by copying `TPL-RPT.md` to `03-pilot/` and renaming it:

RPT-{YYYYMMDD}T{HHmmss}-{YOUR_ACRONYM}-1.00.md

Fill in the six sections:

1. **BLUF**: one paragraph summarising the week. Write this last.
2. **Completed this week**: what did you finish?
3. **In progress**: what are you working on?
4. **Blockers**: what is stopping you?
5. **Decisions made**: what was decided and why?
6. **Next week**: what are your priorities?
7. **References**: Links to LOG files, ZN notes…

The RPT is not a diary. It is a structured record that anyone on the team can read and understand without asking you questions. RPTs aggregate multiple LOGs into a structured summary of a time period.

After the session: commit to Git

As a rule, every session should end with a commit. Consistent commits are what make the project history useful later.

```
git add .
git commit -m "docs: add LOG session 2026-01-14"
git push
```

Follow the Conventional Commits convention for commit messages:

Prefix	Use for
feat	New artefact or template
docs	Log, note, or report entry
fix	Correction to an existing file
chore	Repository maintenance
refactor	Renaming or restructuring

Update the CHANGELOG

At the end of each significant session or week, add an entry to `CHANGELOG.md`:

```
## [0.1.X] — YYYY-MM-DD

### Added
- LOG-20260114T090000-ABC-1.00.md: session description

### Changed
- Updated project-manifest with new deliverable
```

The CHANGELOG is your project diary at the macro level. The LOG files are your project diary at the micro level. Together they give you complete traceability at any scale.

The naming convention

All files follow this pattern (less PROJ and SOW):

{PREFIX}-{YYYYMMDD}T{HHmmss}-{AUTHOR}-{VERSION}.md

Prefix	Document type
LOG	Laboratory log
ZN	Zettel Note
RPT	Weekly report
PROJ	Project document
TPL	Template
SOW	Statement of Work

The timestamp ensures files sort chronologically in any file system. The author acronym makes attribution unambiguous. The version number tracks document evolution.

5. How to Use AI Assistance (Optional)

Read Section 3 and 4 first and make sure your project is already set up before using AI assistance.

Before you start: what AI needs

AI tools do not have persistent memory across sessions unless explicitly provided with context files. Each new conversation may start without access to previous information, so relevant files should be

provided again when needed. The manifests do not give the AI memory. They only provide a structured snapshot that can be reloaded manually. To reconstruct context, you need to attach the following files manually at the start of every session:

Always attach:

- `00-governance/genericYAML/user-manifest-template.yaml`
- `00-governance/genericYAML/governance-manifest-template.yaml`
- `00-governance/genericYAML/project-manifest-template.yaml`

Also attach if available:

- The last RPT or ZN file from `03-pilot/`

Without these files, the AI cannot know who you are, what your project is, or what you did last session. With them, context is reconstructed in seconds.

Choose your AI tool

Any AI assistant works with this framework. The choice depends on your confidentiality requirements:

Tool	Data goes to	Best for
ChatGPT / Claude	External servers	General use
Microsoft Copilot	Microsoft servers	Office 365 environments
Ollama (local)	Your computer only	Confidential research

If your institution has restrictions on sharing data with external services, use Ollama with a local model.

See https://ollama.com for setup instructions.

Session 1: Project Initialisation

Use these prompts in order when starting a new project. Each prompt builds on the context established by the previous one.

Prompt 1 — Generate the SOW

Attached: project-offer.pdf + TPL-PROJ.md

Generate a Statement of Work from the attached project offer. Fill in all fields you can extract directly from the document. Leave unknown fields as [FILL]. Do not invent any data. Output in Markdown.

Prompt 2 — Generate the project manifest

Attached: SOW (already in context) + user-profile.yaml

Generate the project manifest for this project. Use the SOW already in context and my user profile to fill in the fields. Output strictly in YAML. Do not add fields that are not in the template. Leave unknown fields as [FILL].

Prompt 3 — Repository structure

No attachments needed. Context is already active.

Propose a Git repository structure for this project following the QL21 standard (00-governance to 05-journal). Justify any additions specific to this project type. Output as a visual tree.

Prompt 4 — Session closure

No attachments needed. Context is already active.

Generate the session closure artefacts:

1. A Zettel Note (ZN) capturing the key insight from this session. One idea only.
2. A Report (RPT) for project tracking, if you have been working for some days.

Save both to 05-journal/ following the naming convention.

Session 2+: Continuing Work

From the second session onwards, attach only what has changed since the last session.

Attached: user-profile.yaml + project-manifest.yaml + last RPT from 05-journal/

Today I want to work on: [DESCRIBE YOUR TASK]

What AI should never do

AI assistance is for structure and formatting only. Never ask AI to:

- Record experimental observations on your behalf
- Interpret results or draw conclusions
- Write scientific content that requires judgement
- Generate data or parameters you have not measured

The boundary is clear: if it happened in the lab, you write it. If it needs to be structured or formatted, AI can help.

6. How to Maintain the Repository

A repository that is not maintained becomes a liability. This section describes the habits that keep it useful over time.

Commit frequency

Try to commit after every session. It keeps things manageable later.

A good commit message answers three questions: - What changed? (the prefix: feat, docs, fix) - What is it? (the subject: the file or component) - Why? (optional but valuable for significant changes)

```
# Good commit messages
git commit -m "docs: add LOG photolithography session WAF-2026-003"
git commit -m "feat: add ZN diagonal loading insight"
git commit -m "fix: correct YAML indentation in project-manifest"
git commit -m "chore: update CHANGELOG to v0.1.5"

# Bad commit messages
git commit -m "update"
git commit -m "changes"
git commit -m "fix stuff"
```

CHANGELOG maintenance

Update the CHANGELOG at the end of every significant session or week. The CHANGELOG is not a Git log, it is a human-readable summary of what happened and why.

Structure every entry with three sections:

```
## [0.1.X] — YYYY-MM-DD

### Added
- New files or artefacts created this session

### Changed
- Existing files modified and why

### Fixed
```

```
- Errors corrected
```

Keep entries short and specific. The CHANGELOG should be readable in two minutes and give anyone a complete picture of the project status.

README maintenance

Update the README whenever the project status changes significantly. The README is the first thing anyone sees when they open the repository. It should always reflect the current state, not the initial plan.

At minimum, update:
- The deliverables table with current status
- The phase badge if the project has moved to a new phase
- Any new tools or dependencies added

Naming convention discipline

Never rename a file after it has been committed. If a file was committed with the wrong name, create a new file with the correct name, copy the content, and commit it as a correction. Use `git mv` only if you are familiar with Git history and its implications.

If you find yourself creating files with similar names because you are not sure which convention to follow, stop and re-read [[Section 4]]. The naming convention is fixed to keep everything searchable.

Version control for manifests

Governance manifests are versioned documents. When you make a significant change to a manifest, update the version number in the filename and in the manifest header:

- From: sergi-casado-profile-1.0.yaml
- To: sergi-casado-profile-1.1.yaml

If you are working with something like GitHub where you can see the commits, you can delete it, but otherwise no, keep the old version in the repository. Do not delete it. The version history of the manifests is part of the project record.

GitHub Pages deployment

If you want to make your documentation accessible from any device without requiring Git knowledge, enable GitHub Pages:

1. Go to your repository on GitHub
2. Settings → Pages
3. Source: Deploy from a branch
4. Branch: main / root
5. Save

Your repository will be published at:

https://username.github.io/repository-name/

This makes the latest logs and reports accessible from a tablet inside the lab without needing to open a terminal.

The QL21 project documentation site is available as a reference example at:
https://sergicasadoprieto-ctrl.github.io/ql21-toolkit/

7. Project Closure Checklist

Use this checklist when a project is complete, when you are leaving a research group, or when you are handing the project to a new researcher. A closed project should be in a state where someone can continue it without needing constant clarification.

If something cannot be completed, it should be explicitly documented.

Documentation checklist

- ☐ All LOG files are complete and committed. No session without a log.
- ☐ All ZN notes are linked to the relevant LOG files in their references field.
- ☐ All RPT files cover the complete project duration with no gaps.
- ☐ The CHANGELOG reflects every significant version from v0.1.0 to the final version.
- ☐ The README is up to date and reflects the final project status.
- ☐ All deliverables in the README table are marked as Complete or archived.

Governance checklist

- ☐ The project manifest is updated with the actual end date and final status.

- ☐ The user profile reflects the tools and knowledge domains acquired during the project.
- ☐ The SOW is archived with a note indicating whether all objectives were met and which were not.
- ☐ Any deviations from the original scope are documented in the CHANGELOG or a dedicated ZN note.

Repository checklist

- ☐ All files follow the naming convention. No exceptions.
- ☐ The .gitignore excludes OS files, editor configuration files, and any temporary files.
- ☐ The final commit message is clear and descriptive.
- ☐ The repository has been pushed to GitHub and is accessible from the remote URL in the project manifest.

Knowledge transfer checklist

- ☐ All friction points and lessons learned are documented in the final RPT or a dedicated ZN note.
- ☐ Any institutional knowledge that is not in the logs, such as equipment quirks, supplier contacts, or undocumented procedures, has been written down.
- ☐ The repository URL has been shared with the supervisor and any future researchers who will continue this work.

Final commit

```
git add .
git commit -m "chore: project closure - final commit"
git push
```

Archival

After the final commit, tag the repository with the project version:

```
git tag -a v1.0.0 -m "Project closure: final version"
git push origin v1.0.0
```

This creates a permanent, citable snapshot of the repository at the moment of closure.

Quick Reference Card

Action	Command or file
New session	Copy TPL-LOG.md, rename, fill in
New insight	Copy TPL-ZN.md, rename, fill in
End of week	Copy TPL-RPT.md, rename, fill in
Commit	git add . && git commit -m "…"
Push	git push
Update CHANGELOG	Edit CHANGELOG.md, add new entry
Reconstruct context	Read last CHANGELOG entry + last ZN or RPT
Find a file	grep -r "search term" .

Troubleshooting

YAML parsing error in Obsidian Check for missing spaces after colons, tab indentation, or unquoted special characters. See Annex L of the TFG memory for a complete YAML syntax reference.

Git push rejected Run git pull first to integrate remote changes, then push again.

File not found after renaming Never rename committed files outside of Git. Use git mv to rename, or create a new file and commit the old one as deleted.

AI lost context mid-session Reload the governance manifests and the last file. This reconstructs full context in seconds.

Software Resources

The following resources provide the tools referenced in the QL21 workflow:

- Git: https://git-scm.com/downloads
- GitHub: https://github.com
- Obsidian: https://obsidian.md

- Zotero: https://www.zotero.org
- Visual Studio Code: https://code.visualstudio.com
- Pandoc: https://pandoc.org
- Marp: https://marp.app
- Overleaf: https://www.overleaf.com
- Ollama: https://ollama.com

Conclusion

QL21 was developed as a response to a recurring problem in academic and laboratory environments: valuable knowledge is often generated faster than it is documented. The recommendations presented here are based on the workflow used during the development of this project and should be adapted where necessary to fit local practices and research needs.

Repository: https://github.com/sergicasadoprieto-ctrl/tfg-quadern-laboratori-21c